
WEBTECHNOLOGY

(24BECSE304)

Module 2

Topics: Introduction to CSS & the Basics of JavaScript

PREPARED BY:

Prof. AMARSRI.A

Assistant Professor

Dept. of CSE, SNPSU

Module 2

Introduction to CSS: Levels of Style Sheets, Style Specification, Formats, Selector Forms, Property-Value Forms, Font Properties, List Properties, Alignment of Text, Color, The Box Model, Background Images, The and Conflict Resolution.

The Basics of JavaScript: Overview of JavaScript, Object Orientation and JavaScript, General Syntactic Characteristics, Primitives, Operations, and Expressions, Screen Output and Keyboard Input.

Introduction to Cascading Style Sheet

What is CSS?

CSS is a language that applies styles to a HTML document and its elements to change the look And feel and is usually stored in separate .css style which can be re-used for all the web pages. A website is made up of HTML for content plus CSS for appearance.

HTML (Content)	+	CSS (Presentation)	=	WEB PAGE
---------------------------------	----------	-------------------------------------	----------	-----------------

Advantages:

- **CSS saves time** - You can write CSS once and then reuse same sheet in multiple HTML pages. You can define a style for each HTML element and apply it to as many Web pages as you want.
- **Pages load faster** - If you are using CSS, you do not need to write HTML tag attributes every time. Just write one CSS rule of a tag and apply to all the occurrences of that tag. So less code means faster download times.
- **Easy maintenance** - To make a global change, simply change the style, and all elements in all the web pages will be updated automatically.
- **Superior styles to HTML** - CSS has a much wider array of attributes than HTML so you can give far better look to your HTML page in comparison of HTML attributes.
- **Multiple Device Compatibility** - Style sheets allow content to be optimized for more than one type of device. By using the same HTML document, different versions of a website can be presented for handheld devices such as PDAs and cell phones or for printing.
- **Global web standards** - Now HTML attributes are being deprecated and it is being recommended to use CSS. So it's a good idea to start using CSS in all the HTML pages to make them compatible to future browsers.

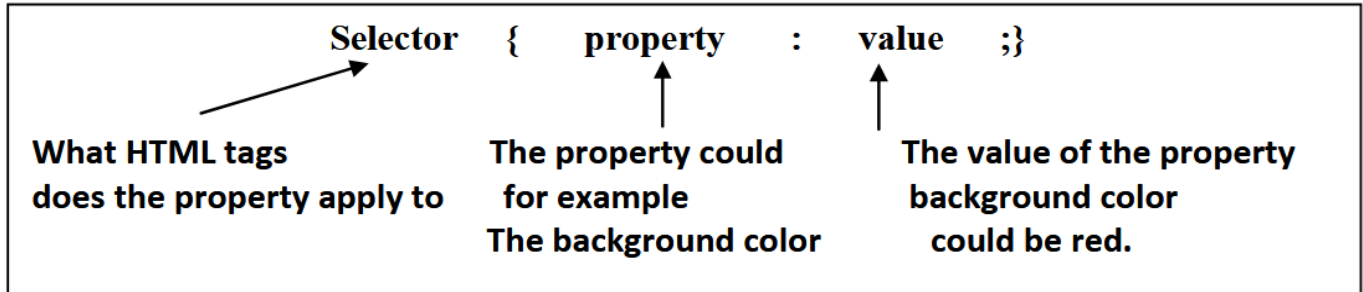
Disadvantages:

- **Browser Compatibility:** Some style sheet features are supported and some are not by the browsers.

Basic CSS Syntax

The CSS syntax consists of a set of rules.

These rules have a 3 parts:



- **Selector:** A selector is an HTML tag at which style will be applied. This could be any tag like <h1> or <table> etc.
- **Property:** A property is a type of attribute of HTML tag. Put simply, all the HTML attributes are converted into CSS properties. They could be color or border etc.
- **Value:** Values are assigned to properties. For example color property can have value either red.

Example: You can define a table border as follows:

```
table{ border :1px; }
```

Here **table** is a selector and **border** is a property and given **value** 1px is the value of that Property.

LEVELS OF STYLESHEET (CSS Inclusion)

There are 3 levels of style sheet used associate css styles with your HTML document.

- 1) Inline style sheet
- 2) Document style sheet
- 3) External style sheet

1) **Inline CSS** - The style attribute:

- Inline style sheet rules will be applied to the content of a single element.
- Inline style sheet rules are specified as the values of the style attribute.
- Here is the generic syntax:

```
<element style="...style rules....">
```

Style rule: is a list of property: value pairs

Example:

Following is the example of inline CSS based on above syntax:

<h1 style = "color: red;"> This is inline CSS </h1>

2) Document Level CSS - The <style> Element:

- Document level style sheet rules will be applied to all the elements available in the document.
- Document level style sheet rules are specified as the content of <style> element.
- This tag is placed inside <head>...</head> tags.
- Here is the generic syntax:

```
<head>
```

```
<style type="text/css" media="projection/screen/tv/print">
```

```
Style Rules
```

```
.....
```

```
</style>
```

```
</head>
```

Where

type="text/css" specifies the style sheet language as a content type. This is required attribute.

media=" screen/tty/tv/projection/handheld/print/all"

Specifies the device the document will be displayed on. Default value is all. This is optional attribute.

3) External CSS - The <link> Element:

- An external style sheet is a separate text file with .css extension.
- You define all the Style rules within this text file and then you can include this file in any HTML document using <link> element.
- External style sheets can apply to the bodies of multiple html documents.
- Here is the generic syntax of including external CSS file:

```
<head>
<link rel="stylesheet" type="text/css" href="..." />
</head>
```

Where

rel="stylesheet" specifies the relationship of the linking document to the current document.

type="text/css" Specifies the style sheet language as a content-type (MIME type).

This attribute is required.

href="filename.css" Specifies the style sheet file having Style rules.

This attribute is a required.

Example:

Consider a simple style sheet file with a name mystyle.css having the following rules:

```
h1 {color: blue; font-size: 34pt; }
```

Now you can include this file mystyle.css in any HTML document as follows:

```
<head>  
<link rel="stylesheet" type="text/css" href="mystyle.css" />  
</head>
```

Style Specification Formats

The format of a style specification depends on the level of style sheet. Inline style specifications appear as values of the style attribute of a tag, the general form of which is as follows:

**style = "property_1 : value_1; property_2 : value_2; ...;
property_n : value_n;"**

Although it is not required, it is recommended that the last property-value pair be followed by a semicolon. Document style specifications appear as the content of a style element within the header of a document, although the format of the specification is quite different from that of inline style sheets. The general form of the content of a style element is as follows:

**<style type = "text/css">
rule_list
</style>**

The type attribute of the **<style>** tag tells the browser the type of style specification, which is **text/css** for CSS.

- Each style rule in a rule list has **two parts**: a **selector**, which indicates the element or elements affected by the rule, and a **list of property**-value pairs.
- The list has the same form as the quoted list for inline style sheets, except that it is delimited by braces rather than double quotes.

**selector {property_1 : value_1; property_2 : value_2; ...;
property_n:value_n;}**

- If a property is given more than one value, those values usually are separated with spaces. For some properties, however, multiple values are separated with commas.
- An external style sheet consists of a list of style rules of the same form as in document style sheets. The <style> tag is not included.

Selector formats

In CSS, a selector is the target element to which each CSS rule is applied.

The different types of selectors are:

- 1) Type selectors
- 2) Class selectors
- 3) ID selectors
- 4) Descendant selectors
- 5) Child selectors
- 6) Universal selectors
- 7) Adjacent sibling selectors
- 8) Attribute selectors
- 9) Pseudo-classes and Pseudo-elements

➤ **The Type Selectors:**

- It is a single element name.
- In this case, the property values in the rule apply to all occurrences of the named elements.

Ex:

```
h1 {font-size:24pt;}
```

2) The Universal Selectors:

The universal selector, denoted by an asterisk (*), applies its style to all elements in the document.

Ex:

```
*{color: red}
```

Makes all elements in the document red.

3) The Descendant Selectors:

Descendant selectors are used to select elements that are descendants of another element in the document tree.

Ex:

```
p em {color: blue;}
```

4) The Class Selectors:

- Class selectors are used to allow different occurrences of the same tag to use different style specifications.
- Class selectors are used to select any XHTML element that has a class attribute.
- A class selector is indicated by a period (.).

Ex:

```
<head>
  p.normal{color:blue;}
</head>
.....
<body>
  <p class="normal">text</p>
</body>
```

5) The ID Selectors:

- ID selector is an individually identified (named) selector to which a specific style is declared.
- ID selectors are created by a character # followed by the selector's name.

Ex:

```
#mypara{color:green;}
.....
<p id="mypara">This is Id selector</p>
```

6) The Child Selectors:

A child selector is used to select an element that is a direct child of another element.

Ex:

```
P>em{font-size:50pt;}
```

7) The Attribute Selectors:

Attribute selectors are used to select elements based on their attribute or attribute values.

Ex:

```
img[src="small.gif"]{border-width:1px; border-style:dotted;}
```

8) Adjacent sibling selectors:

Adjacent sibling selectors will select the siblings immediately following an element.

Ex:

```
em+strong{font-style:italic;}
```

9) Pseudo classes:

- **Pseudo classes are used to add special effects to some selectors.**
- **Pseudo class styles apply when something happens rather than because the target element simply exists.**

Syntax:

Selector: pseudo-class {property: value ;}

The commonly used pseudo classes are:

: First-child =add special style to the first child of another element

: link =add special style to the normal link

: visited=add special style to the visited link

: Active=add special style to the active link

hover: =apply styles when mouse points over the element

focus: =apply styles when element accept input from the keyboard

Ex:

a:visited{color:red;}

a:active{color:green;}

a:link{color:blue;}

input:hover{color:red;}

input:focus{color:green;}

PROPERTY VALUE FORMS

- Every CSS property has a value and value can include some measurements like %,cm,mm etc.,
- **Absolute size:**Sets the text to a specified size
Ex: inches, centimeters, points, and so on,
- **Relative size:**Sets the size relative to surrounding elements
Ex: percentages and em units.

Unit	Description	Example
em	1em is equal to the current font size , 2em means 2 times the size of the current font Ex: if an element is displayed with a font size 12pt, then 2em is 24pt.	P {font-size:1.5em;}
ex	x-height (height of the lower case letter)	P {font-size:2ex;}
px	pixel(a dot on computer screen)	P {font-size:12px;}
in	Define measurements in inches	P {font-size:0.15in;}
cm	Define measurements in centimeters	P {font-size:0.8cm;}
mm	Define measurements in millimeters	P {font-size:4mm;}
pc	Picas(1pica=12points ,1inch=6picas)	P {font-size:1pc;}
pt	Points(1point=1/72 inches)	P {font-size:12pt;}
%	Percentage units are relative to another values 100% takes the current value, 200% doubles it and so on.	P {font-size:200%;}

Color values:

#rrggbb	Standard XHTML red-green-blue 6-digit hexadecimal number. Each hexadecimal code will be preceded by pound sign(#)	P {color:#FFFF00;} [yellow]
#rgb	This is a shorter form of the six-digit notation. In this format ,each digit is replicated to arrive at an equivalent six-digit value; Ex: #F00 becomes FF0000	P {color:#F00;} [red]
rgb(rrr%, ggg%,bbb %)	Percentage representation ,each value being a percentage between 0.0% and 100.0%	P {color:rgb(40%,70%,4%; } }
Rgb(rrr,ggg,bbb)	Decimal representation ,each value being an integer between 0 and 255	P { color:rgb (0,0,255);} [blue]
Aqua,red,black	Using color names	P {color:orange;}

FONT PROPERTIES:

CSS font properties define the font family, boldness, size, and the style of a text.

Font Family:

- Font family is the type of font used for rendering text similar to the fonts used we select in MS-WORD.
- To change the fonts we can use the font-family property with font names as comma separated values.

EX: h1 {font-family:Arial;}

h2 {font-family:Arial,Verdana,"Times New Roman";}

In CSS, there are two types of font family names:

- **Generic family** - a group of font families with a similar look (like "Serif" or "Monospace")
- **Font family** - a specific font family (like "Times New Roman" or "Arial")

Generic family	Description	Examples
Serif	Serif fonts have small lines at the ends on some characters	Times New Roman,Georgia
Sans-serif	"Sans" means without - these fonts do not have the lines at the ends of characters	Arial,Arial Black,Trebuchet MS,Helvetica,Geneva,Verdana
Monospace	All monospace characters have the same width	Courier New,Andale Mono, Lucida Console
Cursive	Cursive scripts emulates handwritten appearance	Apple chancery,,comic Sans,and Zapf chancery, ex: 
fantasy	Fantasy fonts are purely decorative and would be appropriate for headings.	Impact, Western

Font Style: The font-style property is mostly used to specify whether the letters shapes are vertical or slanted.

Value	Description
normal	The text is shown normally
italic	The text is shown in italics(curved letter forms)
oblique	The text is "leaning" (oblique is very similar to italic, it takes the normal font design and just slants it)

Ex:p {font-style:italic;}

Font Size: The font-size property specifies the size of all the text.

Value	Description
xx-small	Sets the font-size to an xx-small size
x-small	Sets the font-size to an extra small size
small	Sets the font-size to a small size
medium	Sets the font-size to a medium size. This is default
large	Sets the font-size to a large size
x-large	Sets the font-size to an extra large size
xx-large	Sets the font-size to an xx-large size
smaller	Sets the font-size to a smaller size than the parent element
larger	Sets the font-size to a larger size than the parent element
length	Sets the font-size to a fixed size in px, cm, etc.
%	Sets the font-size to a percent of the parent element's font size

Ex: h1 {font-size:40px;}
h2 {font-size:150%;}

Font variant:

Specifies whether or not a text should be displayed in a small-caps font.

Value	Description
normal	The browser displays a normal font. This is default
small-caps	The browser displays a small-caps font

EX: p {font-variant: small-caps ;}

Font weights

Font-weight property specifies the thickness of the font.

Value	Description
normal	Defines normal characters. This is default
bold	Defines thick characters
bolder	Defines thicker characters
lighter	Defines lighter characters

EX: p{font-weight: lighter;}

Example program:

```
<!DOCTYPE html>
<!-- fonts.html
An example to illustrate font properties
-->
<html lang = "en">
<head>
<title> Font properties </title>
<meta charset = "utf-8" />
<style type = "text/css">
p.major {font-size: 1.1em;
font-style: italic;
font-family: 'Times New Roman';
}
p.minor {font: 0.9em bold 'Courier New';}
h2 {font-family: 'Times New Roman';
font-size: 2em; font-weight: bold;}
h3 {font-family: 'Courier New'; font-size: 1.5em ;}
</style>
</head>
<body>
<p class = "major">
If a job is worth doing, it's worth doing right.
</p>
<p class = "minor">
Two wrongs don't make a right, but they certainly can get you in a lot of trouble.

</p>
<h2> Chapter 1 Introduction </h2>
<h3> 1.1 The Basics of Computer Networks </h3>
</body>
</html>
```

List Properties

➤ list-style-type

The list-style-type specifies the type of list-item marker in a list.

Value	Description
none	No marker is shown
disc	The marker is a filled circle. This is default for
circle	The marker is a circle
square	The marker is a square
decimal-leading-zero	The marker is a number with leading zeros (01, 02, 03, etc.)
lower-alpha	The marker is lower-alpha (a, b, c, d, e, etc.)
upper-alpha	The marker is upper-alpha (A, B, C, D, E, etc.)
lower-roman	The marker is lower-roman (i, ii, iii, iv, v, etc.)
upper-roman	The marker is upper-roman (I, II, III, IV, V, etc.)

Ex: `ul{list-style-type:square;}`
`ol{list-style-type:upper-roman;}`

➤ list-style-image

The list-style-image property replaces the list-item marker with an image.

Value	Description
url	The path to the image to be used as a list-item marker
none	No image will be displayed. Instead, the list-style-type property will define what type of list marker will be rendered. This is default

Ex: `ul {list-style-image:url('image.jpg');}`

➤ list-style-position

The list-style-position property specifies if the list-item markers should appear inside or outside the content flow.

Value	Description
inside	Indents the marker and the text. The bullets appear inside the content flow
outside	Keeps the marker to the left of the text. The bullets appears outside the content flow. This is default

Ex: `ul{list-style-position:inside;}`

`ul{list-style-position:outside;}`

Example program for unordered list

```
<!-- bullets1 -->
<style type = "text/css">
ul {list-style-type: square;}
</style>
...
<h3> Some Common Single-Engine Aircraft </h3>
<ul>
<li> Cessna Skyhawk </li>
<li> Beechcraft Bonanza </li>
<li> Piper Cherokee </li>
</ul>
```

The following illustrates setting the style for individual list items:

```
<!-- bullets2 -->
<style type = "text/css">
li.disc {list-style-type: disc;}
li.square {list-style-type: square;}
li.circle {list-style-type: circle;}
</style>
...
<h3> Some Common Single-Engine Aircraft </h3>
<ul>
<li class = "disc"> Cessna Skyhawk </li>
<li class = "square"> Beechcraft Bonanza </li>
<li class = "circle"> Piper Cherokee </li>
</ul>
```

The following example illustrates the use of different sequence value types in nested lists:

```
<!DOCTYPE html>
<!-- sequence_types.html
An example to illustrate sequence type styles
-->
<html lang = "en">
<head>
<title> Sequence types </title>
<meta charset = "utf-8" />
<style type = "text/css">
ol {list-style-type: upper-roman;}
ol ol {list-style-type: upper-alpha;}
ol ol ol {list-style-type: decimal;}
</style>
</head>
<body>
<h3> Aircraft Types </h3>
<ol>
<li> General Aviation (piston-driven engines)
<ol>
<li> Single-Engine Aircraft
<ol>
<li> Tail wheel </li>
<li> Tricycle </li>
</ol>
</li>
<li> Dual-Engine Aircraft
<ol>
<li> Wing-mounted engines </li>
<li> Push-pull fuselage-mounted engines </li>
</ol>
</li>
</ol>
```

```
</li>
<li> Commercial Aviation (jet engines)
<ol>
<li> Dual-Engine
<ol>
<li> Wing-mounted engines </li>
<li> Fuselage-mounted engines </li>
</ol>
</li>
<li> Tri-Engine
<ol>
<li> Third engine in vertical stabilizer </li>
<li> Third engine in fuselage </li>
</ol>
</li>
</ol>
</li>
</ol>
</li>
</ol>
</body>
</html>
```

Alignment of Text

➤ **text-align:**

The text-align property specifies the horizontal alignment of text in an element.

Value	Description
left	Left-justified
right	Right-justified
center	Text is centered
justified	Text is both right and left justified

EX: h2 {text-align: left}

➤ **text-decoration:**

The text-decoration property specifies the decoration added to text.

Value	Description
underline	Adds an underline to the text
overline	Adds a line on top of the text
line-through	Adds a line through the middle of the text
blink	Causes the text to blink

EX: h2 {text-decoration: line-through;}

➤ **text-indent:**

The text-indent property specifies the indentation of the first line in a text-block.

EX: p {text-indent:50px;}

➤ **text-transform:**

The text-transform property controls the capitalization of text.

Value	Description
capitalize	Capitalizes the first letter in a word
uppercase	Makes the entire word uppercase
lowercase	Makes the entire word lower case
none	No transform is performed

EX: p{text-transform:capitalize;}

Color:

The color property specifies the color of text (foreground color).

We can specify the color values in various formats:

Format	Syntax	example
Hex code	#rrggbb	P {color:#ffff00;}
Short Hex Code	#rgb	P {color:#6b7;}
rgb%	rgb(rrr%,ggg%,bbb%)	P {color:rgb(50%,50%,50%);}
Rgb absolute	Rgb(rrr,ggg,bbb)	p {color:rgb(0,0,255);}
keyword	Aqua,red,black	P {color:orange;}

Ex: body {color: red ;}

h1 {color: #00ff00 ;}

p>em {color: rgb (0, 0,255) ;}

background-color:

The background-color property is used to set the background color.

Ex: body {background-color: red ;}

h1 {background-color: rgb (0, 0,255) ;}

CSS Box Model

- The term “box model” is about CSS based layouts and design.
- The box model is a term used when referring to the rectangular boxes placed around every element in the web page.
- An XHTML element can be considered a box, and so the box model applies to all XHTML elements.
- The box model is the specification that defines how a box and its attributes relate to each other.



The 4 different parts of the Box model are:

1. Content:

The innermost part of the box is the content, such as “<h1>”, “”, “<p>”...etc. The width and height property defines the width and height of the element.

Ex:

```
P {  
width: 100px;  
height: 50px ;  
}
```

2. Padding:

The padding defines the space between the content and the border.

The top, right, bottom, and left padding can be changed independently using separate properties.

Ex:

```
p{  
padding-top:25px;  
padding-bottom:25px;  
padding-right:50px;  
padding-left:50px;  
}
```

A shorthand padding property can also be used, to change all paddings at once.

Ex: P {padding: 10px 5px 15px 20px ;}

3. Border:

The middle layer in the box model is the element's border.

The space used by the border in the box model is the thickness of the border.

CSS border has following 4 properties:

- ☐ border-style
- ☐ border-Width
- ☐ border -Color
- ☐ border - Individual sides

The border-style property sets the style of an element's four borders.

Ex:

1) P

```
{  
border-top-style:dotted;  
border-right-style:solid;  
border-bottom-style:dotted;  
border-left-style:solid;  
}
```

2)

```
P {  
border-style:solid;  
border-color:red;  
border-width:5px;  
}
```

4. Margin:

- The space just outside the border is margin.
- The margin is completely invisible.
- The top, right, bottom, and left margin can be changed independently using separate properties.

Ex: P

```
{  
margin-top:100px;  
margin-bottom:100px;  
margin-right:50px;  
margin-left:50px;  
}
```

A shorthand margin property can also be used, to change all margins at once.

Ex: P {margin: 25px 50px 75px 100px ;}

CSS and <div> tags

<div> Tag

<div>(Division) tag divides the content into individual sections .

Div is a block-level container, meaning that there is a line feed after the </div> tag.

Ex: <div style="color:#00FF00">

<h3>This is a header</h3>

<p>This is a paragraph.</p>

</div>

** Tag**

 tag is used for grouping and applying styles to inline elements.

The span is an inline-level container, meaning that there is no line feed after the tag.

They can only contain text or data in the document like STRONG and EM tags.

Ex :<p>My mother has

light blue eyes

</p>

Background –images

The background-image property is used to add a background image to an element.

Ex: body {background-image: url ('siley.gif');}

Background-repeat

- By default, the image is repeated both in x- and y- directions so it covers the entire element.
- The background-repeat property specifies whether a background image repeats itself.
- We can specify no-repeat, repeat-x, and repeat-y.

Ex: body (background-repeat: repeat-y; }

Background-position

- The position of the image is specified by the background-position property.
- The values can be: A combination of [top, center, bottom] and [left, center, right].

Ex: body {background-position: center ;}

Conflict Resolution

- When there are two different values for the same property on the same element in a document, there is an obvious conflict that the browser must resolve.
- We have considered only one way this conflict can occur: when style sheets at two or more levels specify different values for the same property on the same element.
- This particular kind of conflict is resolved by the precedence of the **three different levels of style sheets**.
- Inline style sheets have precedence over document and external style sheets, and document style sheets have precedence over external style sheets.

The following shows an external style sheet and an HTML document that has three paragraph elements.

- The first paragraph uses the external style sheet to determine the font size.
- In the second, the document-level style sheet specifies the font size.
- In the third, although the document-level style sheet applies, the inline style sheet overrides it to specify the font size.

/* cstyle.css - an external style sheet for use with cascade.html */

p {font-size: 0.8em;}

<!DOCTYPE html>

<!-- cascade.html An example to illustrate the three levels of style sheets -->

<html lang = "en">

<head>

<title> Style sheet levels </title>

<meta charset = "utf-8" />

```
<link rel = "stylesheet" type = "text/css"
href = "cstyle.css" />
<style type = "text/css">
p.docstyle {font-size: 1.2em;}
</style>
</head>
<body>
<p>
Now is the time</p>
<p class = "docstyle">
for all good men
</p>
<p class = "docstyle" style = "font-size: 1.6em">
to come to the aid
</p>
</body>
</html>
```

Property-value conflicts can occur in several other ways. For example, a conflict may occur within a single style sheet. Consider the following style specifications, which are next to each other in the same document level style sheet:

```
h3 {color: blue;}
body h3 {color: red;}
```

Both of these specifications apply to all h3 elements in the body of the document.

Next, all of the available specifications (those from style sheets, those from the user, and those from the browser) are sorted by origin and weight in accordance with the following list of rules, in which the first has the highest precedence:

1. Important declarations with user origin
2. Important declarations with author origin
3. Normal declarations with author origin
4. Normal declarations with user origin

5. Any declarations with browser (or other user agent) origin

If there are conflicts after this first sorting takes place, the next step in their resolution is a sort by specificity. This sort is based on the following rules, in which the first has the highest precedence:

1. Id selectors
2. Class and pseudo class selectors
3. Contextual selectors (more element type names means that they are more specific)
4. Universal selectors

The whole sorting process that is used to resolve style specification conflicts is called **the cascade**. The name is apropos because the rules apply the lowest priority styles first and then cascade progressively to those with higher priorities.

Java script:

“JavaScript is a cross-platform, object-based scripting language invented specifically for use in web browsers to make websites more dynamic and attractive.”

- A script is a program or sequence of instructions that is interpreted or carried out by another program.
- Some languages: VBScript, JavaScript, Jscript and ECMA Script.
- Scripts can be executed on client or the server.

Client Side Scripting	Server Side Scripting
Runs on the user's computer i.e. Browser interprets the script	Runs on the Web server
Visible to users if they view the HTML source	Not visible to user

- JavaScript is mainly used as a client side scripting language.
- JavaScript was originated with Netscape and it is invented by Brendan Eich.
- JavaScript was initially called as Live Script.
- Later in 1995, after an agreement with Sun Microsystems, Live Script was re-named JavaScript, to Leverage the popularity of Java.

Uses of JavaScript:

JavaScript as a programming language has many uses. A non-exhaustive list of its uses is given below:

1. JavaScript is used for validating user input on client-side.
2. JavaScript is used for file handling and database connectivity on the server-side.
3. JavaScript can be used as an alternative to applets.
4. JavaScript is used to implement programming on client-side web documents.
5. JavaScript can be used to implement event driven programming.
6. Along with DOM, JavaScript can be used to develop DHTML (Dynamic HTML).

Object Orientation and JavaScript

- JavaScript is not an object-oriented programming language.
- Rather, it is an object-based language.
- JavaScript does not have classes.
- Its objects serve both as objects and as models of objects.
- Without classes, JavaScript cannot have class-based inheritance, which is supported in object-oriented languages such as C++ and Java.
- It does support a technique that can be used to simulate some of the aspects of inheritance. This is done with the prototype object; thus, this form of inheritance is called prototype-based inheritance

JavaScript Objects

- In JavaScript, objects are collections of properties, which correspond to the members of classes in Java and C++.
- Each property is either a data property or a function or method property.
- Data properties appear in two categories:
 1. Primitive values
 2. References to other objects

We prefer to call subprograms that are called through objects methods and subprograms that are not called through objects functions.

- JavaScript uses non-object types for some of its simplest types; these non-object types are called primitives.
- Primitives are like the simple scalar variables of non-object-oriented languages such as C, C++, Java, and JavaScript all have both primitives and objects.
- All objects in a JavaScript program are indirectly accessed through variables. Such variables are like references in Java.
- All primitive values in JavaScript are accessed directly—these are like the scalar types in Java and C++. These are often called value types.

For example:

If **myCar** is a variable referencing an object that has the property engine, the engine property can be referenced with **myCar.engine**.

- A JavaScript object appears, both internally and externally, as a list of property–value pairs.
- The properties are names; the values are data values or functions.
- All functions are objects and are referenced through variables.
- The collection of properties of a JavaScript object is dynamic: Properties can be added or deleted at any time during execution.

General Syntactic Characteristics

JavaScript is often written in another file which is saved with .js extension. But it can be also embedded with HTML code in a web document. JavaScript is embedded in a web document using the **<script>** tags as shown below:

```
<script type="text/javascript">  
---JavaScript code here---  
</script>
```

The **<script>** tags can be written in the head section and as well in the body section for multiple times. They will be interpreted by the JavaScript interpreter in the order in which they are written in the web document. The script which has to be executed once is written in the body section and other scripts are written in the head section.

Another way of specifying JavaScript is to write it in a separate file. Below **example** demonstrates external JavaScript:

script.js (JavaScript file)

document.write("Welcome to JavaScript");

hello.html (Web document)

<html>

<head><title>First JavaScript</title></head>

<body>

<script type="text/javascript" src="script.js"></script>

</body>

</html>

While specifying JavaScript in another file (**like script.js**), it should be remembered that there is a need to specify that file name as a value to the **src** attribute of **<script>** tag. The type attribute specifies the MIME type. The closing script tag **</script>** is mandatory even though there is no content for the script element.

Primitives, Operations, and Expressions

The primitive data types, operations, and expressions of JavaScript are similar to those of other common programming languages.

Primitives:

JavaScript provides five scalar primitive types: Number, Boolean, String, Undefined, Null and two compound primitive types Array and Object. Even though JavaScript supports object-orientation features, it still supports these scalar primitive data types simply because of performance reasons. Maintenance of primitive values is much faster than objects.

JavaScript also provides object versions known as wrapper objects for the primitive types Number, Boolean and String namely, Number, Boolean and String. All primitive values are stored on stack whereas objects are stored in heap.

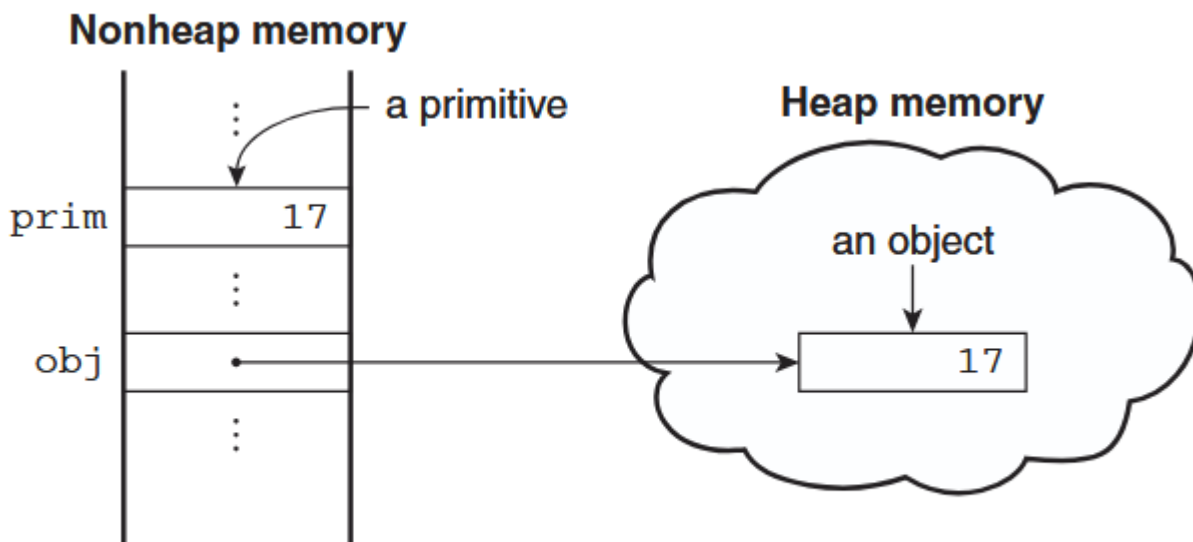


Figure 1 Primitives and objects

Literals:

A literal is a primitive constant value. In JavaScript all numeric values are of type Number. Internally all numerical values are stored as double-precision floating point values. This is why numeric values are often called as numbers.

In JavaScript integer values are a sequence of digits with no decimal points and floating point numbers contains digits and a decimal point or digits and an exponent or both. The exponent can be represented using e or E. Hexadecimal numbers can be represented by preceding a integer value with 0x or 0X.

A string literal is a sequence of characters enclosed in single quotes (') or double quotes ("). A null string can be denoted with a pair of single quotes (") or a pair of double quotes ("").

The only value of type Null is the reserved word null, which indicates no value and the only value of type Undefined is undefined which is not a reserved word like null. If a variable is not explicitly declared or is not assigned, it is said to be null. If a variable is explicitly declared and is not assigned, it is said to be undefined.

The only possible values for Boolean type are true and false.

Variables:

A variable is named memory location to store data. JavaScript is a loosely typed (or dynamically typed) language i.e., there is no need to declare the data type of a variable. The type of a variable is dynamically decided at runtime based on the value assigned. A variable can be assigned any of the five primitive type values or it can refer an object. A variable can be declared implicitly as shown below:

```
pi = 3.14; //The data type of pi is Number
```

A variable can be declared explicitly as shown below:

```
var pi = 3.14; //The data type of pi is Number
```

```
var name = "John"; //The data type of name is String
```

Comments:

Every programming language supports comments through which a programmer can provide details like author name, creation date and purpose of the script. Apart from these details, comments allow programmers to write explanations or reviews about their script or elements in the script.

A single line comment in JavaScript starts with //. Everything followed by // in that line is treated as a comment.

A single line or multi-line comment starts with /* and ends with */. Everything in between these two markers is treated as a comment and will be ignored by the JavaScript interpreter.

Operators:

Numeric Operators:

As any typical programming language, JavaScript provides the numeric operators + (addition), - (subtraction), * (multiplication), / (division), % (modulo), ++ (increment) and -- (decrement).

If the increment appears before a variable, it is known as pre increment. Otherwise, if it appears after a variable, it is known as post increment. Similarly for decrement operator we have pre decrement and post

decrement.

The precedence rules of a language specify which operator should be evaluated when there are multiple operators in an expression which belong to different precedence levels. The associativity rules specify which operator should be evaluated when an expression contains multiple operations of the same precedence level. The precedence and associativity rules for numeric operators are shown below:

Operators	Associativity
++, --, unary -, unary +	Right-to-Left
*, /, %	Left-to-Right
+, -	Left-to-Right

String Concatenation Operator:

Unlike C, strings in JavaScript are not stored or treated as arrays. Strings in JavaScript are unit scalar values. Two strings can be concatenated with each other using the + operator.

For example, if a variable first holds the string value "Java" then the result of the following expression will be JavaScript:

first + "Script"

The type of Operator:

The typeof operator accepts a single operand either a value or a variable and returns its type as a string value. It returns "**number**", "**string**" and "**boolean**" for values or variables of the type Number, String and Boolean respectively. It returns "**object**" for values or variables of the type object or null. For a variable which is not assigned a value, the type of operator returns "**undefined**".

The type of operator can be used in situations where there is a need to be absolute about the type of a value of variable before proceeding further. The type of operator either may include parentheses for the operand or may not. So, both type of variable and type of(variable) are same.

Assignment Statements:

Like any other typical programming language, JavaScript supports assignment statements. An assignment statement contains the assignment operator or any one of the compound assignment operators as shown below:

x += 1; is same as:

x = x + 1;

Type Conversion:

The mechanism of converting a value or variable from one type to another type is known as type conversion.

There two types of type conversion: **implicit type conversion and explicit type conversion.**

➤ **Implicit Type Conversion:**

Type conversion which is performed automatically by the JavaScript interpreter is known as implicit type conversion or coercion. Implicit type conversion may take place when the expected type is different from the actual type of the value given from the user or read from a file etc. For example, consider the following expression:

"John" + 123

It can be observed that in the above expression, + is a concatenation operator. So, even though the second operand is a number, it will be converted (implicit conversion) to a string and will be concatenated with "John". Now, consider the following example:

8 * "2"

The * operator is only applicable on numbers. So, the second operand will be converted from a string to a number and the resultant value will be 16.

Boolean false will be equal to 0 and true will be equal to 1 when converted to number. Both null and undefined will become false when converted to Boolean. A number 0 will be converted to false and any other number will be converted to true, when it is converted to a Boolean.

➤ **Explicit Type Conversion:**

Sometimes there is a need for the programmer to force type conversion. Such conversion specified by the programmer is known as explicit type conversion or type casting.

For example, a number can be casted to a string using the String constructor as shown below:

var str = String(5);

We can also cast a number to String using the toString() method which provides an advantage of specifying the base of the number. Consider the following expressions:

var x = 5;

var y = x.toString();

var z = x.toString(2); //2 specifies the base of the result. 5 in base 2 is 101

In the above expressions, y will have the string "5" and z will have the string "101". A string can be

casted to a number using the Number constructor as shown below:

```
var x = Number("10");
```

The above code works only when the supplied string parameter does not contain any other characters after the number. To convert a string which contains a number at any position, we can use `parseInt( )` and `parseFloat( )` methods.

Strings:

A string is a collection of characters. Most of the times in scripts, there is a need to work with strings. JavaScript provides various properties and methods to work with String objects. Whenever a String property or method is used on a string value, it will be coerced to a String object.

One most frequently used property on String objects is `length`. The `length` property gives the number of characters in the given string. Consider the following example:

```
var str = "Hello World";
```

```
var len = str.length;
```

The value stored in the `len` variable will be 11 which is the number of characters in the string. Frequently used methods on String objects are given below:

Method	Description
<code>charAt(pos)</code>	Returns the character at specified position
<code>indexOf(char)</code>	Returns the position of the given character in the string
<code>substring(startpos, endpos)</code>	Returns the substring in between the specified positions
<code>toLowerCase()</code>	Returns the string with all characters converted to lower case
<code>toUpperCase()</code>	Returns the string with all characters converted to upper case

Math Object:

The Math object provides a number of properties and methods to work with Number values. Among the methods there are `sin` and `cos` for trigonometric functions, `floor` and `round` for truncating and rounding the given numbers and `max` for returning the maximum of given numbers.

Number Object:

The number object contains a collection of useful properties and methods to work with Numbers. The properties include: `MAX_VALUE` (represents largest number that is available), `MIN_VALUE` (represents smallest number that is available), `NaN` (represents Not a Number), `POSITIVE_INFINITY` (special value

to represent infinity), `NEGATIVE_INFINITY` (special value to represent negative infinity) and `PI` (represents value of π).

To test whether the value in a variable is NaN or not, there is a method `isNaN( var )`, which returns true if the value in the specified variable is NaN and false otherwise. To convert a Number to a String, use the method `toString( )` as shown below:

```
var cost = 250;
```

```
var output = cost.toString( );
```

Date Object:

At times you there will be a need to access the current date and time and also past and future date and times. JavaScript provides support for working with dates and time through the Date object.

Date object is created using the new operator and one of the Date's constructors. Current date and time can be retrieved as shown below:

```
var today = new Date( );
```

Below are some of the frequently used methods available on a Date object:

Method	Description
<code>toLocaleString</code>	A string of the date information
<code>getDate</code>	The day of the month
<code>getMonth</code>	The month of the year, as a number in the range of 0 to 11
<code>getDay</code>	The day of a week, as a number in the range 0 to 6
<code>getFullYear</code>	The year
<code>getTime</code>	Number of milliseconds since Jan 1, 1970
<code>getHours</code>	Number of the hour, as a number in the range 0 to 23
<code>getMinutes</code>	Number of the minute, as a number in the range 0 to 59
<code>getSeconds</code>	Number of the second, as a number in the range 0 to 59
<code>getMilliseconds</code>	Number of milliseconds, as a number in the range of 0 to 999

Screen Output and Keyboard Input

JavaScript models the HTML/XHTML document as the document object and the browser window which displays the HTML/XHTML document as the window object. The document object contains a method named `write` to output text and data on to the document as shown below:

```
document.write("Welcome to JavaScript");
```

To display multiple lines of text using the write method, include the HTML tag `<br />` as a part of the string parameter as shown below:

```
document.write("Welcome<br />to<br />JavaScript");
```

The output of the above code on the document will be as shown below:

```
Welcome  
to  
JavaScript
```

Although the document object has another method named `writeln` which appends a new line (`\n`), the output of both **write** and **writeln** will be same as the browser ignores the new line characters.

The window object is the default object in JavaScript. So, whenever a property or method on window object is to be referenced, there is no need to mention window object explicitly.

There are three methods available on the window object to perform input/output namely `alert` and `confirm` for output and `prompt` for input. The `alert` method is used to display a dialog box which shows the provided text or data as output along with a OK button. The `alert` method can be used as shown below:

```
alert("Sum of 10 and 20 is: " + (10 + 20));
```

The `alert` method allows the new line character (`\n`) for displaying text in multiple lines and does not support HTML tags like `<br />` etc.

The `confirm` method displays a dialog box which displays the string provided as output along with two buttons labelled **OK and Cancel**. This method returns a Boolean value `true` when the user clicks OK button and `false` when user clicks Cancel button. The `confirm` method can be used as shown below:

```
var status = confirm("Do you want to proceed?");
```

The `prompt` method is used to accept string input from the user.

This accepts two parameters. First parameter is a string that will be displayed on the dialog box and the second string is the default input string in case the user doesn't provide any input. The `prompt` method displays a string along with a textbox and two buttons labeled OK and Cancel.

Prompt box can be created as shown below:

```
var a = prompt("Enter a value: ", "");
```